

Interface Inteligente para Consulta de Arquivos CSV

Relatório Técnico -- Desafio 4

I2A2 -- Instituto de Inteligência Artificial Aplicada

Agentes Inteligentes e LLMs

Framework de agentes: LangChain (create_agent / LangGraph)

Modelo de linguagem: Groq -- openai/gpt-oss-20b

Interface: Streamlit (3 abas)

1. Framework escolhido

O agente foi construído com **LangChain**, usando a API atual de agentes da versão 1.x (`langchain.agents.create_agent`), que orquestra o loop de tool-calling internamente via LangGraph. O modelo de linguagem é o **openai/gpt-oss-20b**, servido pela **Groq** (via `langchain-groq`). A interface das três abas (carga de dados, consulta e arquitetura) foi feita em **Streamlit**, o que também permite publicação gratuita no Streamlit Community Cloud.

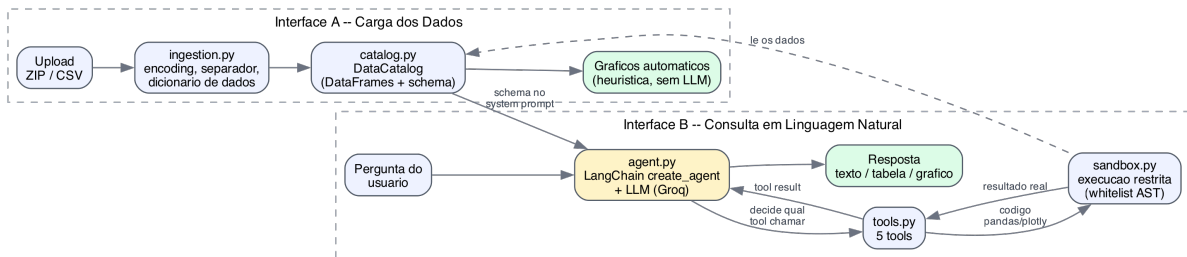
Por que Groq e não uma LLM proprietária: testamos primeiro o Google Gemini (`gemini-2.5-flash`) e depois o Groq com `llama-3.3-70b-versatile` -- ambos com tier gratuito, mas com cotas diárias baixas demais para uso real em sala de aula: o Gemini limita 20 requisições/dia por projeto, e o Llama 70B na Groq estourava sua cota de *tokens*/dia (100k) em poucas perguntas, já que cada pergunta do agente consome ao menos 2 chamadas (decisão de tool + resposta final) e o system prompt carrega o schema completo do catálogo a cada turno. Testamos então um modelo menor (`llama-3.1-8b-instant`), mas as respostas finais perderam os números concretos exigidos pelo prompt (violando a regra "nunca responda de memória"). O `openai/gpt-oss-20b` na Groq manteve a qualidade das respostas equivalente à do modelo 70B, com uma cota de tokens/dia bem mais folgada -- adequado para múltiplos integrantes do grupo testarem sem custo.

2. Arquitetura da solução

A aplicação é dividida em módulos de responsabilidade única, sem lógica de negócio misturada na interface:

Arquivo	Responsabilidade
<code>app.py</code>	Interface Streamlit: Aba A (upload), Aba B (chat), Aba C (arquitetura)
<code>src/csvagent/ingestion.py</code>	Extraí o ZIP; detecta encoding e separador por arquivo; identifica o dicionário de dados; normaliza colunas com inteiros gigantes (ex: chave de acesso de NF-e) para evitar <code>OverflowError</code> do <code>pyarrow</code>
<code>src/csvagent/catalog.py</code>	Catálogo em memória: <code>DataFrames</code> + schema + dicionário de dados; gera o resumo textual para o prompt
<code>src/csvagent/autocharts.py</code>	Até 2 gráficos genéricos por tabela (categoria + evolução no tempo), sugeridos por heurística sobre o schema -- sem chamar o LLM
<code>src/csvagent/sandbox.py</code>	Execução restrita (whitelist de AST) do código <code>pandas/plotly</code> gerado pelo agente
<code>src/csvagent/tools.py</code>	5 tools do agente + registro do trace de decisão para a UI
<code>src/csvagent/agent.py</code>	Monta o agente: system prompt + LLM (Groq) + tools

Diagrama do fluxo completo



Módulos em verde não dependem da LLM (heurística pura); o módulo em amarelo (`agent.py`) é onde o LangChain + Groq efetivamente decidem o que fazer. O mesmo diagrama é exibido ao vivo na Aba 3 ("Arquitetura") do app, renderizado via `st.graphviz_chart`.

Interface A -- Carga dos dados

O usuário envia um .zip com um ou mais CSVs (ou CSVs soltos, sem precisar compactar) e, opcionalmente, um dicionário de dados. O pipeline de ingestão: (1) extrai os membros do zip ignorando lixo de SO (`__MACOSX`, arquivos ocultos); (2) para cada arquivo .csv/.txt, detecta o encoding via `chardet` e o separador via `csv.Sniffer` com fallback manual (testa `;` e `\t`); (3) classifica cada arquivo como DADOS ou DICIONÁRIO DE DADOS por heurística de nome e, na ausência disso, pelo formato do cabeçalho; (4) monta um catálogo em memória com todas as tabelas e o dicionário associado; (5) sugere até 2 gráficos automáticos por tabela via heurística de schema (sem custo de LLM).

Interface B -- Consulta em linguagem natural

O catálogo (schema + amostra + dicionário) é injetado no system prompt do agente. A cada pergunta do usuário, o agente decide livremente qual sequência de tools chamar. O resultado de cada chamada de tool é registrado (nome da tool, código gerado, resultado real) e exibido num expander "Ver raciocínio do agente".

Interface C -- Arquitetura

Terceira aba, adicionada para tornar a arquitetura auto-explicativa dentro do próprio app: mostra o mesmo diagrama Graphviz reproduzido acima, ao vivo.

Sandbox de execução

O código `pandas/plotly` escrito pelo LLM não é executado livremente: passa antes por um validador de AST (`sandbox.py`) que bloqueia `import`, `eval/exec`, acesso a atributos `__dunder__`, definição de função/classe e loops `while`. Os únicos nomes expostos ao código são `pd`, `px/go` e o dicionário `tables` com cópias dos DataFrames carregados. Não é isolamento de processo/SO -- é uma defesa em profundidade adequada ao escopo educacional do desafio.

3. Descrição do agente e das ferramentas (tools)

A solução usa um único agente (arquitetura enxuta, conforme recomendado pelo enunciado), equipado com 5 tools especializadas:

- **listar_tabelas** -- Lista as tabelas carregadas, com número de linhas e colunas. Usada quando o agente não tem certeza do nome exato da tabela.
- **descrever_tabela** -- Mostra schema detalhado (nome/tipo de cada coluna + descrição do dicionário de dados), estatísticas de colunas numéricas e uma amostra de linhas.
- **valores_frequentes** -- Retorna os valores mais frequentes de uma coluna categórica, com contagem -- útil para explorar categorias antes de filtrar/agrupar.
- **executar_pandas** -- Executa uma consulta pandas real sobre as tabelas (agregações, somas, contagens, rankings, filtros) e retorna o resultado real, nunca um número "chutado".
- **gerar_grafico** -- Gera um gráfico Plotly (px/go) a partir das tabelas, quando o usuário pede um gráfico ou quando uma visualização ajuda a resposta.

O system prompt instrui explicitamente o modelo a nunca responder de memória: toda resposta quantitativa deve vir da execução real de uma tool, e a interface nunca deve conter tags de imagem markdown inventadas (uma alucinação observada nos testes com este modelo, corrigida via instrução explícita no prompt).

4. Fluxo de funcionamento da aplicação

- 1 Usuário envia um .zip ou CSV(s) na Aba A.
- 2 ingestion.py extrai, detecta encoding/separador, classifica cada arquivo (dado vs. dicionário) e normaliza colunas problemáticas.
- 3 catalog.py monta o catálogo em memória e o resumo textual do schema; autocharts.py sugere até 2 gráficos genéricos por heurística.
- 4 Ao abrir a Aba B, agent.py monta o agente: system prompt (catálogo) + LLM (Groq) + as 5 tools.
- 5 Usuário faz uma pergunta em linguagem natural no chat.
- 6 O agente (LangGraph por baixo) decide, em loop, quais tools chamar -- cada chamada roda de fato sobre os dados (via sandbox.py) e o resultado real volta para o agente.
- 7 O agente produz uma resposta final em português, com números concretos.
- 8 A interface exibe a resposta, qualquer tabela/gráfico gerado, e um expander com o trace completo de tools chamadas.
- 9 A Aba C exibe, a qualquer momento, o diagrama da arquitetura completa.

5. Perguntas realizadas e respostas obtidas

Testes realizados com um CSV sintético de notas fiscais (5 linhas: id_nf, fornecedor, produto, categoria, valor, data_emissao) acompanhado de um dicionário de dados no mesmo .zip, processado end-to-end pela aplicação real (upload -> ingestão -> agente -> resposta) com o modelo de produção (Groq openai/gpt-oss-20b). Para cada pergunta, o trace real do agente (tool chamada + código executado) é mostrado abaixo do texto -- exatamente o que aparece no expander "Ver raciocínio do agente" da Interface B.

P1. Qual fornecedor recebeu o maior valor no período?

Resposta do agente: O fornecedor que recebeu o maior valor no período foi Fornecedor A, com um total de R\$ 601,25.

tool: executar_pandas

```
resultado = tables["notas"].groupby("fornecedor")["valor"].sum().sort_values(ascending=False).head(1)
```

P2. Qual produto apresentou o maior volume comprado (em quantidade de notas)?

Resposta do agente: O produto que apareceu em mais notas fiscais é Parafuso, com 2 notas registradas.

tool: executar_pandas

```
resultado = tables["notas"].groupby("produto").size().reset_index(name="count").sort_values("count", ascending=False).head(1)
```

P3. Qual foi o total gasto em cada mês?

Resposta do agente: Total gasto por mês: 2024-01: R\$ 350,50 -- 2024-02: R\$ 651,00 -- 2024-03: R\$ 80,00. (o agente também gerou um gráfico de barras automaticamente, por iniciativa própria)

tool: executar_pandas

```
resultado = tables["notas"].assign(mes=pd.to_datetime(tables["notas"].data_emissao).dt.to_period('M')).groupby("mes")["valor"].sum().reset_index()
```

tool: gerar_grafico

```
fig = px.bar(resultado, x='mes', y='valor', labels={'mes':'Mês','valor':'Total Gasto (R$)'}, title='Total gasto por mês')
```

P4. Quais foram os cinco maiores fornecedores?

Resposta do agente: Fornecedor A (R\$ 601,25), Fornecedor B (R\$ 330,00), Fornecedor C (R\$ 150,25). O agente observou corretamente que o dataset só tem 3 fornecedores, então não há 5 para listar.

tool: executar_pandas

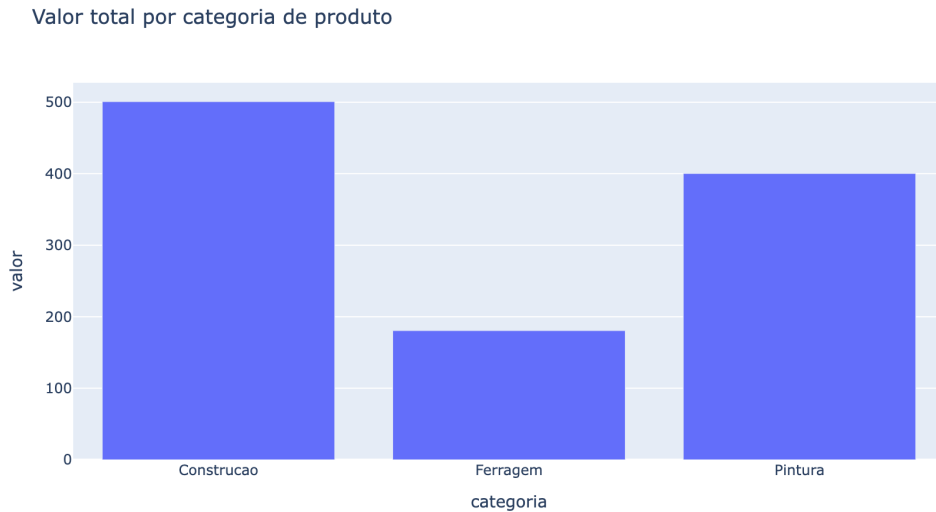
```
resultado = tables["notas"].groupby("fornecedor")["valor"].sum().sort_values(ascending=False).head(5)
```

P5. Gere um gráfico do valor total por categoria de produto.

Resposta do agente: "Gráfico gerado com sucesso e será exibido para o usuário."

tool: gerar_grafico

```
fig = px.bar(tables["notas"].groupby("categoria")["valor"].sum().reset_index(),  
x="categoria", y="valor", title="Valor total por categoria de produto")
```



6. Limitações conhecidas (MVP)

- Um único dicionário de dados é aplicado a todas as tabelas do ZIP (cenário mais comum quando cabeçalho/itens de uma mesma nota compartilham glossário); não há suporte a um dicionário por tabela.
- O sandbox impede `while` e algumas funções perigosas, mas não há timeout de execução -- um `for` sobre uma tabela muito grande ainda pode ser lento.
- Sem suporte a múltiplos ZIPs na mesma sessão (recarregar substitui o catálogo carregado).
- O tier gratuito da Groq tem cota diária de tokens (bem mais folgada que a de LLMs proprietárias, mas ainda finita) -- para uso contínuo em sala de aula com muitos integrantes, recomenda-se cada um usar sua própria chave, ou considerar o tier pago.
- Os gráficos automáticos da Aba A são heurísticos (baseados em tipo de coluna e cardinalidade) -- não substituem uma análise dirigida por pergunta, servem como visão geral inicial dos dados.

7. Como executar

```
uv sync
# criar .env com GROQ_API_KEY=... (ou colar a chave na barra lateral do app)
uv run streamlit run app.py
```